SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science & Engineering

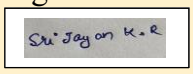
ASSESSMENT TOOL 3 – OUTPUT PREDICTION

Course Code	**CSA0314**	**Course Title**	**Data Structures**
CO Assessed	**CO1**	**Assessment**	**Assessment Tool 3: Output Prediction**
Weightage	**35% of CIA**	**Total Marks**	**100**
CO1 Statement	***Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity.* (BL2, BL3)**		
Student Name	**K.R.Sri Jayan**	**Reg. No.**	**1921260271**
Section/Batch	**Slot C / 2026**	**Date**	**10-10-2026**

Code of Conduct:

I , K.R. Sri Jayan(1921260271) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Output Accuracy	30%	Predicts all program outputs correctly, including exact values and output sequence.	Predicts most outputs correctly with minor errors.	Predicts some outputs correctly but makes significant errors.	Fails to predict most program outputs correctly.	
Conceptual Understanding	25%	Demonstrates excellent understanding of ADT, arrays, recursion, data structures, and C programming concepts.	Demonstrates good understanding with minor conceptual errors.	Demonstrates partial understanding with several conceptual errors.	Demonstrates inadequate understanding of fundamental concepts.	
Logical Tracing and Execution Flow	20%	Traces loops, recursive calls, array updates, and function execution accurately and systematically.	Traces execution correctly with a few minor mistakes.	Shows incomplete tracing with errors in execution flow.	Unable to trace program execution logically.	
Time and Space Complexity Analysis	15%	Correctly identifies and justifies best-, average-, and worst-case complexity and auxiliary space requirements.	Identifies most complexities correctly with minor errors.	Identifies some complexities but provides incomplete or incorrect reasoning.	Unable to identify or explain algorithm complexity.	
C Syntax and Runtime Behavior	10%	Correctly interprets C expressions, variable updates, function calls, and undefined behavior.	Understands most C execution rules with minor mistakes.	Shows limited understanding of execution rules and expression evaluation.	Misinterprets fundamental C language rules and program behavior.	

Question 1: Recursive Operation Counter

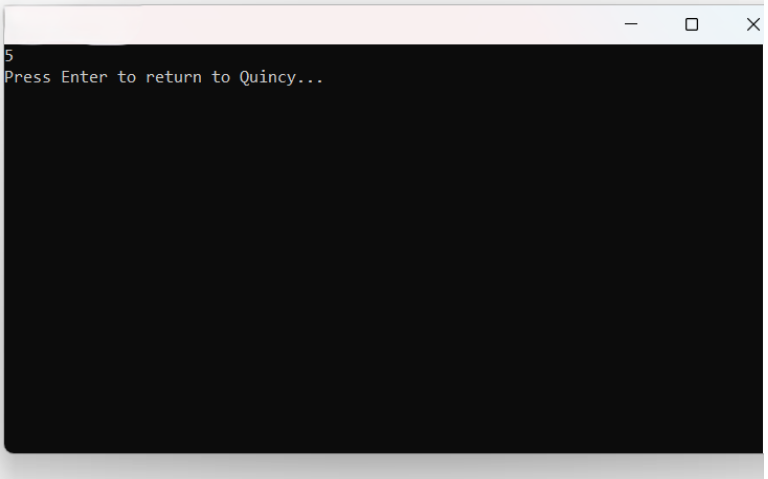
```c
#include <stdio.h>

int count = 0;

void fun(int n)
{
    if (n <= 1)
        return;

    count++;
    fun(n / 2);
}

int main()
{
    fun(32);
    printf("%d", count);
    return 0;
}
```



Question 2: Best, Average and Worst Cases in Linear Search

```c
#include <stdio.h>

int search(int a[], int n, int key)
{
    int i, count = 0;

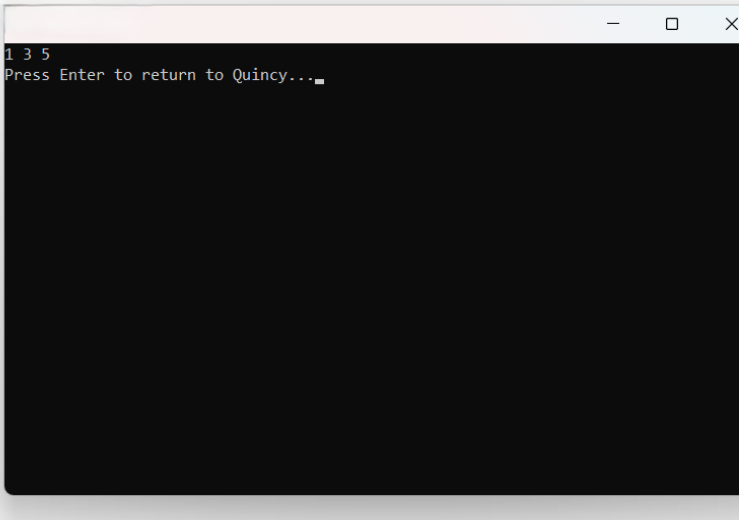
    for (i = 0; i < n; i++) {
        count++;
        if (a[i] == key)
            break;
    }

    return count;
}

int main()
{
    int a[] = {2, 4, 6, 8, 10};

    printf("%d %d %d",
           search(a, 5, 2),
           search(a, 5, 6),
           search(a, 5, 10));

    return 0;
}
```



Question 3: Space Usage in Recursion

```c
#include <stdio.h>

int fun(int n)
{
    if (n == 0)
        return 0;

    return 1 + fun(n - 1);
}

int main()
{
    printf("%d", fun(5));
    return 0;
}
```

```
5
Press Enter to return to Quincy...
```

Question 4: Comparing Two Algorithms

```c
#include <stdio.h>

int main()
{
    int i, j;
    int count1 = 0, count2 = 0;

    for (i = 0; i < 5; i++)
    {
        count1++;
    }

    for (i = 0; i < 5; i++)
    {
        for (j = 0; j < 5; j++)
        {
            count2++;
        }
    }

    printf("%d %d", count1, count2);

    return 0;
}
```

```
5 25
Press Enter to return to Quincy...
```

Question 5: Integrated Array, Recursion and Complexity Challenge

```c
#include <stdio.h>

int calls = 0;

int fun(int a[], int n)
{
    calls++;

    if (n == 1)
    {
        return a[0];
    }

    return a[n - 1] + fun(a, n - 1);
}

int main()
{
    int a[] = {1, 2, 3, 4, 5};

    printf("%d ", fun(a, 5));
    printf("%d", calls);

    return 0;
}
```

